Banco de ejercicios Python para coloquio

Sistemas y Web · 50 ejercicios con enunciado y resolución

Ejercicios sobre listas, tuplas, diccionarios, condicionales, `for` y `while`, pensados para que cada estudiante resuelva 2 sorteados. Todos corren con Python 3 sin librerías externas; las resoluciones y salidas de este documento se generaron ejecutando el código en Python 3.12.

Bloque	Ejercicios	Tipo
A	1 a 18	Situaciones de Sistemas: instalación desatendida, logs, usuarios, servicios
B	19 a 34	Situaciones Web: formularios, carrito, rutas, sesiones, JSON
C	35 a 50	Código que no corre o da mal resultado: probar, explicar y corregir

Sugerencia para PEDCO. Cargar cada ejercicio como pregunta tipo Ensayo en una categoría por bloque. En el cuestionario, agregar 1 pregunta aleatoria de A o B y 1 aleatoria de C. Moodle sortea por estudiante de forma independiente, así que puede haber alguna repetición; si tiene que ser estrictamente distinto para todos, conviene asignar los pares a mano.

Criterio de corrección sugerido. El código corre (40 %), resuelve lo pedido con las estructuras indicadas (40 %), y el estudiante explica su solución o el error encontrado (20 %).

Las resoluciones son una forma posible de resolver cada ejercicio; cualquier solución que cumpla el enunciado con las estructuras pedidas es válida.

Bloque A: situaciones de Sistemas (1 a 18)

Cada ejercicio simula una tarea de administración con datos en memoria, sin tocar el sistema real.

1. Instalación desatendida: paquetes faltantes

Temas: listas, for, if.

Un script de instalación desatendida tiene que asegurar que estén todos los paquetes requeridos. Mostrá qué paquetes faltan y generá el comando en una sola línea: `sudo apt-get install -y paquete1 paquete2 ...`. Si no falta ninguno, mostrá "Sistema listo".

Datos de partida

```
requeridos = ["git", "curl", "python3", "apache2", "mysql-server", "ufw"]
instalados = ["curl", "python3", "nano", "ufw"]
```

Resolución

```
requeridos = ["git", "curl", "python3", "apache2", "mysql-server", "ufw"]
instalados = ["curl", "python3", "nano", "ufw"]
faltantes = []
for p in requeridos:
    if p not in instalados:
        faltantes.append(p)
if faltantes:
    print("Faltan:", faltantes)
    print("sudo apt-get install -y " + " ".join(faltantes))
else:
    print("Sistema listo")
```

Salida esperada

```
Faltan: ['git', 'apache2', 'mysql-server']
sudo apt-get install -y git apache2 mysql-server
```

2. Requisitos mínimos del equipo

Temas: diccionarios, tuplas, condicionales.

Antes de instalar, el script verifica el equipo. Los requisitos mínimos están en una tupla (RAM en GB, disco libre en GB). Informá "APTO" o "NO APTO" y, si no es apto, qué requisito no cumple. Además, si el sistema operativo no empieza con "Ubuntu", mostrá una advertencia.

Datos de partida

```
requisitos = (4, 20)
equipo = {"hostname": "lab2-pc07", "ram_gb": 2, "disco_libre_gb": 35, "so": "Debian 12"}
```

Resolución

```
requisitos = (4, 20)
equipo = {"hostname": "lab2-pc07", "ram_gb": 2, "disco_libre_gb": 35, "so": "Debian 12"}
ram_min, disco_min = requisitos
apto = True
if equipo["ram_gb"] < ram_min:
    print("RAM insuficiente:", equipo["ram_gb"], "GB")
    apto = False
if equipo["disco_libre_gb"] < disco_min:
    print("Disco insuficiente:", equipo["disco_libre_gb"], "GB")
    apto = False
print(equipo["hostname"], "APTO" if apto else "NO APTO")
if not equipo["so"].startswith("Ubuntu"):
    print("Advertencia: el SO no es Ubuntu")
```

Salida esperada

```
RAM insuficiente: 2 GB
lab2-pc07 NO APTO
Advertencia: el SO no es Ubuntu
```

3. Análisis de un log

Temas: listas, diccionarios, for, split().

Contá cuántas líneas hay de cada nivel (INFO, WARNING, ERROR) usando un diccionario. Al final, mostrá solo los mensajes de ERROR.

Datos de partida

```
log = [
    "2026-11-03 10:15 INFO Servicio apache2 iniciado",
    "2026-11-03 10:16 WARNING Disco al 75%",
    "2026-11-03 10:20 ERROR Falló conexión a mysql",
    "2026-11-03 10:21 INFO Reintentando conexión",
    "2026-11-03 10:22 ERROR Timeout en mysql",
    "2026-11-03 10:30 INFO Backup finalizado",
]
```

Resolución

```
log = [
    "2026-11-03 10:15 INFO Servicio apache2 iniciado",
    "2026-11-03 10:16 WARNING Disco al 75%",
    "2026-11-03 10:20 ERROR Falló conexión a mysql",
    "2026-11-03 10:21 INFO Reintentando conexión",
    "2026-11-03 10:22 ERROR Timeout en mysql",
    "2026-11-03 10:30 INFO Backup finalizado",
]
conteo = {}
errores = []
for linea in log:
    partes = linea.split(" ", 3)
    nivel = partes[2]
    conteo[nivel] = conteo.get(nivel, 0) + 1
    if nivel == "ERROR":
        errores.append(partes[3])
print(conteo)
for e in errores:
    print("ERROR:", e)
```

Salida esperada

```
{'INFO': 3, 'WARNING': 1, 'ERROR': 2}
ERROR: Falló conexión a mysql
ERROR: Timeout en mysql
```

4. Reintentos de conexión

Temas: while, listas, condicionales.

Un script intenta conectarse a un servidor de paquetes como máximo 5 veces. La lista `respuestas` simula el resultado de cada intento. Usá un `while` que muestre "Intento N: falló" o "Intento N: conectado" y se detenga al conectar. Si se agotan los intentos, mostrá "Instalación abortada". Probá también con una lista de 5 `False`.

Datos de partida

```
respuestas = [False, False, False, True, False]
max_intentos = 5
```

Resolución

```
respuestas = [False, False, False, True, False]
max_intentos = 5
intento = 0
conectado = False
while intento < max_intentos and not conectado:
    if respuestas[intento]:
        print("Intento", intento + 1, ": conectado")
        conectado = True
    else:
        print("Intento", intento + 1, ": falló")
    intento += 1
if not conectado:
    print("Instalación abortada")
```

Salida esperada

```
Intento 1 : falló
Intento 2 : falló
Intento 3 : falló
Intento 4 : conectado
```

5. Usuarios con acceso a consola

Temas: lista de tuplas, for, condicionales compuestos.

Cada tupla representa una línea de `/etc/passwd` simplificada: (usuario, uid, shell). Listá los usuarios "humanos" (uid mayor o igual a 1000) que tienen shell `/bin/bash`, y contá cuántos son.

Datos de partida

```
usuarios = [
    ("root", 0, "/bin/bash"), ("daemon", 1, "/usr/sbin/nologin"),
    ("ana", 1000, "/bin/bash"), ("luis", 1001, "/bin/sh"),
    ("www-data", 33, "/usr/sbin/nologin"), ("caro", 1002, "/bin/bash"),
]
```

Resolución

```
usuarios = [
    ("root", 0, "/bin/bash"), ("daemon", 1, "/usr/sbin/nologin"),
    ("ana", 1000, "/bin/bash"), ("luis", 1001, "/bin/sh"),
    ("www-data", 33, "/usr/sbin/nologin"), ("caro", 1002, "/bin/bash"),
]
cantidad = 0
for nombre, uid, shell in usuarios:
    if uid >= 1000 and shell == "/bin/bash":
        print(nombre)
        cantidad += 1
print("Total:", cantidad)
```

Salida esperada

```
ana
caro
Total: 2
```

6. Inventario del laboratorio

Temas: diccionario de diccionarios, for, condicionales.

Mostrá las PCs que necesitan actualización de RAM (menos de 8 GB) y las que tienen Windows. Al final, informá el promedio de RAM del laboratorio.

Datos de partida

```
inventario = {
    "pc01": {"ram": 8, "so": "Ubuntu"},
    "pc02": {"ram": 4, "so": "Windows"},
    "pc03": {"ram": 16, "so": "Ubuntu"},
    "pc04": {"ram": 4, "so": "Debian"},
    "pc05": {"ram": 8, "so": "Windows"},
}
```

Resolución

```
inventario = {
    "pc01": {"ram": 8, "so": "Ubuntu"},
    "pc02": {"ram": 4, "so": "Windows"},
    "pc03": {"ram": 16, "so": "Ubuntu"},
    "pc04": {"ram": 4, "so": "Debian"},
    "pc05": {"ram": 8, "so": "Windows"},
}
suma = 0
for pc, datos in inventario.items():
    if datos["ram"] < 8:
        print(pc, "necesita más RAM")
    if datos["so"] == "Windows":
        print(pc, "tiene Windows")
    suma += datos["ram"]
print("Promedio de RAM:", suma / len(inventario))
```

Salida esperada

```
pc02 necesita más RAM
pc02 tiene Windows
pc04 necesita más RAM
pc05 tiene Windows
Promedio de RAM: 8.0
```

7. Estado de servicios

Temas: diccionarios, listas, condicionales.

Armá una lista con los servicios caídos y mostrala. Si entre los caídos está "ssh", mostrá "ALERTA CRÍTICA: sin acceso remoto". Si no hay ninguno caído, mostrá "Todos los servicios OK".

Datos de partida

```
servicios = {"apache2": "activo", "mysql": "inactivo", "ssh": "inactivo", "cron": "activo", "uf
    w": "activo"}
```

Resolución

```
servicios = {"apache2": "activo", "mysql": "inactivo", "ssh": "inactivo", "cron": "activo", "uf
    w": "activo"}
caidos = []
for nombre, estado in servicios.items():
    if estado != "activo":
        caidos.append(nombre)
if not caidos:
    print("Todos los servicios OK")
else:
    print("Caídos:", caidos)
    if "ssh" in caidos:
        print("ALERTA CRÍTICA: sin acceso remoto")
```

Salida esperada

```
Caídos: ['mysql', 'ssh']
ALERTA CRÍTICA: sin acceso remoto
```

8. Uso de disco

Temas: lista de tuplas, if/elif/else.

Clasificá cada punto de montaje: OK (menos de 70 %), ADVERTENCIA (70 a 89 %) o CRÍTICO (90 % o más). Mostrá una línea por partición con el formato `/var -> 92% CRÍTICO`.

Datos de partida

```
particiones = [("/", 45), ("/home", 71), ("/var", 92), ("/boot", 89), ("/tmp", 10)]
```

Resolución

```
particiones = [("/", 45), ("/home", 71), ("/var", 92), ("/boot", 89), ("/tmp", 10)]
for punto, uso in particiones:
    if uso >= 90:
        estado = "CRÍTICO"
    elif uso >= 70:
        estado = "ADVERTENCIA"
    else:
        estado = "OK"
    print(f"{punto} -> {uso}% {estado}")
```

Salida esperada

```
/ -> 45% OK
/home -> 71% ADVERTENCIA
/var -> 92% CRÍTICO
/boot -> 89% ADVERTENCIA
/tmp -> 10% OK
```

9. Nombres de host para un laboratorio

Temas: for, range, listas, cadenas.

Generá la lista de nombres de host para el laboratorio 3 con 20 equipos, con formato `lab3-pc01` hasta `lab3-pc20` (siempre con dos dígitos). Mostrá la lista y cuántos nombres se generaron. Extra: que el número de laboratorio y la cantidad de equipos se pidan con `input()`.

Resolución

La solución fija lab = 3 y cantidad = 20; para el extra, reemplazar por int(input(...)).

```
lab = 3
cantidad = 20
hosts = []
for i in range(1, cantidad + 1):
    hosts.append(f"lab{lab}-pc{i:02d}")
print(hosts)
print("Generados:", len(hosts))
```

Salida esperada

```
['lab3-pc01', 'lab3-pc02', 'lab3-pc03', 'lab3-pc04', 'lab3-pc05', 'lab3-pc06', 'lab3-pc07', 'la
    b3-pc08', 'lab3-pc09', 'lab3-pc10', 'lab3-pc11', 'lab3-pc12', 'lab3-pc13', 'lab3-pc14', 'la
    b3-pc15', 'lab3-pc16', 'lab3-pc17', 'lab3-pc18', 'lab3-pc19', 'lab3-pc20']
Generados: 20
```

10. Menú de administración de paquetes

Temas: while, input, listas, if/elif.

Hacé un menú que se repita hasta elegir 0: 1) listar paquetes, 2) agregar paquete (sin permitir repetidos), 3) quitar paquete (avisando si no existe), 0) salir.

Datos de partida

```
paquetes = ["git", "curl", "vim"]
```

Resolución

```
paquetes = ["git", "curl", "vim"]
opcion = ""
while opcion != "0":
    print("1) Listar  2) Agregar  3) Quitar  0) Salir")
    opcion = input("Opción: ")
    if opcion == "1":
        for p in paquetes:
            print("-", p)
    elif opcion == "2":
        nuevo = input("Paquete: ")
        if nuevo in paquetes:
            print("Ya existe")
        else:
            paquetes.append(nuevo)
    elif opcion == "3":
        quitar = input("Paquete: ")
        if quitar in paquetes:
            paquetes.remove(quitar)
        else:
            print("No existe")
    elif opcion != "0":
        print("Opción inválida")
print("Saliendo")
```

Salida de ejemplo (entradas: 1, 2 nano, 2 git, 3 zzz, 0)

```
1) Listar  2) Agregar  3) Quitar  0) Salir
Opción: - git
- curl
- vim
1) Listar  2) Agregar  3) Quitar  0) Salir
Opción: Paquete: 1) Listar  2) Agregar  3) Quitar  0) Salir
Opción: Paquete: Ya existe
1) Listar  2) Agregar  3) Quitar  0) Salir
Opción: Paquete: No existe
1) Listar  2) Agregar  3) Quitar  0) Salir
Opción: Saliendo
```

11. Validación de dirección IP

Temas: cadenas, listas, for, condicionales.

Antes de configurar la red, el script valida la IP. Una IP es válida si tiene 4 partes separadas por punto, cada parte es numérica y está entre 0 y 255. Probá con todas las de la lista e informá cuáles son válidas.

Datos de partida

```
ips = ["192.168.1.10", "10.0.0.256", "172.16.5", "8.8.8.8", "192.168.a.1"]
```

Resolución

```
ips = ["192.168.1.10", "10.0.0.256", "172.16.5", "8.8.8.8", "192.168.a.1"]
for ip in ips:
    partes = ip.split(".")
    valida = len(partes) == 4
    for p in partes:
        if not p.isdigit() or int(p) > 255:
            valida = False
    print(ip, "válida" if valida else "inválida")
```

Salida esperada

```
192.168.1.10 válida
10.0.0.256 inválida
172.16.5 inválida
8.8.8.8 válida
192.168.a.1 inválida
```

12. Orden de instalación por dependencias

Temas: diccionarios, listas, while, for.

Cada paquete solo se puede instalar cuando todas sus dependencias ya están instaladas. Usá un `while` que recorra los pendientes y vaya "instalando" los que se puedan, hasta que no quede ninguno. Mostrá el orden final de instalación.

Datos de partida

```
dependencias = {
    "wordpress": ["apache2", "php", "mysql-server"],
    "php": ["apache2"],
    "apache2": [],
    "mysql-server": [],
    "phpmyadmin": ["php", "mysql-server"],
}
```

Resolución

El break hace que el for vuelva a recorrer los pendientes desde el principio después de cada instalación.

```
dependencias = {
    "wordpress": ["apache2", "php", "mysql-server"],
    "php": ["apache2"],
    "apache2": [],
    "mysql-server": [],
    "phpmyadmin": ["php", "mysql-server"],
}
instalados = []
pendientes = list(dependencias.keys())
while pendientes:
    for paquete in pendientes:
        listo = True
        for dep in dependencias[paquete]:
            if dep not in instalados:
                listo = False
        if listo:
            instalados.append(paquete)
            pendientes.remove(paquete)
            break
print("Orden:", instalados)
```

Salida esperada

```
Orden: ['apache2', 'php', 'mysql-server', 'wordpress', 'phpmyadmin']
```

13. Rotación de backups

Temas: listas, slicing, sort, for.

La política es conservar solo los 3 backups más recientes. Ordená la lista, mostrá cuáles se conservan y cuáles se "borrarían". (Pista: el formato de fecha AAAA-MM-DD permite ordenar como texto.)

Datos de partida

```
backups = ["backup_2026-10-01.tar.gz", "backup_2026-11-01.tar.gz", "backup_2026-09-15.tar.gz",
           "backup_2026-10-20.tar.gz", "backup_2026-11-04.tar.gz"]
```

Resolución

```
backups = ["backup_2026-10-01.tar.gz", "backup_2026-11-01.tar.gz", "backup_2026-09-15.tar.gz",
           "backup_2026-10-20.tar.gz", "backup_2026-11-04.tar.gz"]
backups.sort()
conservar = backups[-3:]
borrar = backups[:-3]
print("Se conservan:", conservar)
for b in borrar:
    print("Se borraría:", b)
```

Salida esperada

```
Se conservan: ['backup_2026-10-20.tar.gz', 'backup_2026-11-01.tar.gz', 'backup_2026-11-04.tar.g
    z']
Se borraría: backup_2026-09-15.tar.gz
Se borraría: backup_2026-10-01.tar.gz
```

14. Configuración por defecto y personalizada

Temas: diccionarios, for, in.

Un instalador tiene una configuración por defecto y el usuario puede sobrescribir valores. Generá la configuración final combinando ambas. Si el usuario puso una clave que no existe en la configuración por defecto, no la agregues y mostrá "Clave desconocida: ...".

Datos de partida

```
por_defecto = {"puerto": 80, "idioma": "es", "zona_horaria": "America/Argentina/Buenos_Aires",
    "firewall": True}
usuario = {"puerto": 8080, "firewal": False, "idioma": "es"}
```

Resolución

```
por_defecto = {"puerto": 80, "idioma": "es", "zona_horaria": "America/Argentina/Buenos_Aires",
    "firewall": True}
usuario = {"puerto": 8080, "firewal": False, "idioma": "es"}
final = por_defecto.copy()
for clave, valor in usuario.items():
    if clave in final:
        final[clave] = valor
    else:
        print("Clave desconocida:", clave)
print(final)
```

Salida esperada

```
Clave desconocida: firewal
{'puerto': 8080, 'idioma': 'es', 'zona_horaria': 'America/Argentina/Buenos_Aires', 'firewall':
    True}
```

15. Auditoría de puertos abiertos

Temas: listas, diccionarios, condicionales.

Recorré los puertos abiertos del servidor. Si el puerto está en el diccionario de conocidos, mostrá su servicio; si no, marcalo como SOSPECHOSO y agregalo a una lista. Al final mostrá cuántos sospechosos hay.

Datos de partida

```
abiertos = [22, 80, 443, 3306, 4444, 8080, 31337]
conocidos = {22: "ssh", 80: "http", 443: "https", 3306: "mysql", 8080: "http-alt"}
```

Resolución

```
abiertos = [22, 80, 443, 3306, 4444, 8080, 31337]
conocidos = {22: "ssh", 80: "http", 443: "https", 3306: "mysql", 8080: "http-alt"}
sospechosos = []
for p in abiertos:
    if p in conocidos:
        print(p, "->", conocidos[p])
    else:
        print(p, "-> SOSPECHOSO")
        sospechosos.append(p)
print("Sospechosos:", len(sospechosos), sospechosos)
```

Salida esperada

```
22 -> ssh
80 -> http
443 -> https
3306 -> mysql
4444 -> SOSPECHOSO
8080 -> http-alt
31337 -> SOSPECHOSO
Sospechosos: 2 [4444, 31337]
```

16. Contraseñas temporales para cuentas nuevas

Temas: listas, for, cadenas, condicionales.

Cada contraseña debe tener al menos 8 caracteres, al menos un dígito y al menos una mayúscula. Mostrá para cada una si es válida y, si no, qué le falta.

Datos de partida

```
claves = ["Comahue2026", "abc123", "PasswordSegura", "sistemas99", "Lab3pc07"]
```

Resolución

```
claves = ["Comahue2026", "abc123", "PasswordSegura", "sistemas99", "Lab3pc07"]
for c in claves:
    faltas = []
    if len(c) < 8:
        faltas.append("8 caracteres")
    tiene_digito = False
    tiene_mayus = False
    for car in c:
        if car.isdigit():
            tiene_digito = True
        if car.isupper():
            tiene_mayus = True
    if not tiene_digito:
        faltas.append("un dígito")
    if not tiene_mayus:
        faltas.append("una mayúscula")
    print(c, "válida" if not faltas else "le falta: " + ", ".join(faltas))
```

Salida esperada

```
Comahue2026 válida
abc123 le falta: 8 caracteres, una mayúscula
PasswordSegura le falta: un dígito
sistemas99 le falta: una mayúscula
Lab3pc07 válida
```

17. Cola de tareas programadas

Temas: lista de tuplas, sort, while, pop.

Ordená las tareas por hora y "ejecutalas" una por una con un `while` que saque la primera de la lista hasta vaciarla, mostrando `[02:00] Ejecutando: backup`.

Datos de partida

```
tareas = [("03:30", "actualizar paquetes"), ("02:00", "backup"), ("05:00", "reiniciar apache2")
    , ("02:45", "limpiar /tmp")]
```

Resolución

```
tareas = [("03:30", "actualizar paquetes"), ("02:00", "backup"), ("05:00", "reiniciar apache2")
    , ("02:45", "limpiar /tmp")]
tareas.sort()
while tareas:
    hora, tarea = tareas.pop(0)
    print(f"[{hora}] Ejecutando: {tarea}")
```

Salida esperada

```
[02:00] Ejecutando: backup
[02:45] Ejecutando: limpiar /tmp
[03:30] Ejecutando: actualizar paquetes
[05:00] Ejecutando: reiniciar apache2
```

18. Bloqueo de IPs por intentos fallidos

Temas: listas, diccionarios, for, split().

A partir de un log de autenticación, contá los intentos fallidos por IP. Las IPs con más de 3 fallos se agregan a una lista negra. Mostrá el conteo y la lista negra (similar a lo que hace fail2ban).

Datos de partida

```
auth = [
    "FAIL 10.0.0.5", "OK 10.0.0.2", "FAIL 10.0.0.5", "FAIL 200.45.1.9",
    "FAIL 10.0.0.5", "FAIL 200.45.1.9", "FAIL 10.0.0.5", "OK 10.0.0.5", "FAIL 186.12.3.4",
]
```

Resolución

```
auth = [
    "FAIL 10.0.0.5", "OK 10.0.0.2", "FAIL 10.0.0.5", "FAIL 200.45.1.9",
    "FAIL 10.0.0.5", "FAIL 200.45.1.9", "FAIL 10.0.0.5", "OK 10.0.0.5", "FAIL 186.12.3.4",
]
fallos = {}
for linea in auth:
    resultado, ip = linea.split()
    if resultado == "FAIL":
        fallos[ip] = fallos.get(ip, 0) + 1
lista_negra = []
for ip, cant in fallos.items():
    if cant > 3:
        lista_negra.append(ip)
print(fallos)
print("Lista negra:", lista_negra)
```

Salida esperada

```
{'10.0.0.5': 4, '200.45.1.9': 2, '186.12.3.4': 1}
Lista negra: ['10.0.0.5']
```

Bloque B: situaciones Web (19 a 34)

Simulan la lógica del lado del servidor de una aplicación web, con datos como los que llegarían de un formulario o una API.

19. Validación de formulario de registro

Temas: diccionarios, condicionales, listas.

Los datos llegan de un formulario. Validá: nombre, email y edad son obligatorios (no vacíos); el email debe contener "@" y un "." después; la edad debe ser 18 o más. Guardá todos los errores en una lista y mostralos; si no hay, mostrá "Registro exitoso".

Datos de partida

```
formulario = {"nombre": "Sofía", "email": "sofia.gmail.com", "edad": "17", "ciudad": ""}
```

Resolución

```
formulario = {"nombre": "Sofía", "email": "sofia.gmail.com", "edad": "17", "ciudad": ""}
errores = []
for campo in ("nombre", "email", "edad"):
    if formulario.get(campo, "") == "":
        errores.append(f"El campo {campo} es obligatorio")
email = formulario["email"]
if "@" not in email or "." not in email[email.find("@"):]:
    errores.append("Email inválido")
if formulario["edad"].isdigit() and int(formulario["edad"]) < 18:
    errores.append("Debe ser mayor de 18")
if errores:
    for e in errores:
        print("-", e)
else:
    print("Registro exitoso")
```

Salida esperada

```
- Email inválido
- Debe ser mayor de 18
```

20. Carrito de compras

Temas: lista de tuplas, for, condicionales.

Cada tupla es (producto, precio unitario, cantidad). Mostrá el subtotal de cada línea y el total. Si el total supera $50.000, aplicá 10 % de descuento; si además hay 5 o más unidades en total, el envío es gratis (si no, cuesta $3.500).

Datos de partida

```
carrito = [("Teclado", 18000, 1), ("Mouse", 9500, 2), ("Pad", 3000, 3)]
```

Resolución

Con estos datos el total es $46.000 (no hay descuento) y hay 6 unidades, así que el envío es gratis.

```
carrito = [("Teclado", 18000, 1), ("Mouse", 9500, 2), ("Pad", 3000, 3)]
total = 0
unidades = 0
for producto, precio, cant in carrito:
    subtotal = precio * cant
    print(producto, subtotal)
    total += subtotal
    unidades += cant
if total > 50000:
    total = total * 0.9
    print("Descuento 10% aplicado")
envio = 0 if unidades >= 5 else 3500
print("Total:", total, "Envío:", envio, "A pagar:", total + envio)
```

Salida esperada

```
Teclado 18000
Mouse 19000
Pad 9000
Total: 46000 Envío: 0 A pagar: 46000
```

21. Pedido contra stock

Temas: diccionario de diccionarios, lista de tuplas, condicionales.

Procesá cada ítem del pedido: si hay stock suficiente, descontalo y sumá al total; si no, rechazá ese ítem con un mensaje. Si el producto no existe en el catálogo, informá "Producto inexistente". Mostrá el stock final.

Datos de partida

```
catalogo = {
    "remera": {"precio": 12000, "stock": 5},
    "buzo": {"precio": 30000, "stock": 1},
    "gorra": {"precio": 8000, "stock": 0},
}
pedido = [("remera", 2), ("buzo", 2), ("gorra", 1), ("campera", 1)]
```

Resolución

```
catalogo = {
    "remera": {"precio": 12000, "stock": 5},
    "buzo": {"precio": 30000, "stock": 1},
    "gorra": {"precio": 8000, "stock": 0},
}
pedido = [("remera", 2), ("buzo", 2), ("gorra", 1), ("campera", 1)]
total = 0
for producto, cant in pedido:
    if producto not in catalogo:
        print(producto, ": Producto inexistente")
    elif catalogo[producto]["stock"] >= cant:
        catalogo[producto]["stock"] -= cant
        total += catalogo[producto]["precio"] * cant
        print(producto, ": OK")
    else:
        print(producto, ": sin stock suficiente")
print("Total:", total)
print(catalogo)
```

Salida esperada

```
remera : OK
buzo : sin stock suficiente
gorra : sin stock suficiente
campera : Producto inexistente
Total: 24000
{'remera': {'precio': 12000, 'stock': 3}, 'buzo': {'precio': 30000, 'stock': 1}, 'gorra': {'pre
    cio': 8000, 'stock': 0}}
```

22. Router simple con error 404

Temas: diccionarios, for, in.

Un servidor web relaciona cada ruta con una página. Para cada URL pedida, mostrá `200 OK -> inicio.html` o `404 Not Found -> /ruta`. Al final, mostrá cuántas respuestas fueron 404.

Datos de partida

```
rutas = {"/": "inicio.html", "/contacto": "contacto.html", "/productos": "productos.html", "/lo
    gin": "login.html"}
pedidas = ["/", "/productos", "/admin", "/contacto", "/carrito", "/login"]
```

Resolución

```
rutas = {"/": "inicio.html", "/contacto": "contacto.html", "/productos": "productos.html", "/lo
    gin": "login.html"}
pedidas = ["/", "/productos", "/admin", "/contacto", "/carrito", "/login"]
errores = 0
for url in pedidas:
    if url in rutas:
        print("200 OK ->", rutas[url])
    else:
        print("404 Not Found ->", url)
        errores += 1
print("Total 404:", errores)
```

Salida esperada

```
200 OK -> inicio.html
200 OK -> productos.html
404 Not Found -> /admin
200 OK -> contacto.html
404 Not Found -> /carrito
200 OK -> login.html
Total 404: 2
```

23. Login con 3 intentos

Temas: diccionarios, while, input, condicionales.

Pedí usuario y contraseña. Si son correctos, mostrá "Bienvenido/a, usuario". El usuario tiene 3 intentos; al agotarlos, mostrá "Cuenta bloqueada". Distinguí "Usuario inexistente" de "Contraseña incorrecta".

Datos de partida

```
cuentas = {"admin": "Adm1n!", "caro": "web2026", "invitado": "1234"}
```

Resolución

```
cuentas = {"admin": "Adm1n!", "caro": "web2026", "invitado": "1234"}
intentos = 0
ingreso = False
while intentos < 3 and not ingreso:
    usuario = input("Usuario: ")
    clave = input("Contraseña: ")
    if usuario not in cuentas:
        print("Usuario inexistente")
    elif cuentas[usuario] != clave:
        print("Contraseña incorrecta")
    else:
        print("Bienvenido/a,", usuario)
        ingreso = True
    intentos += 1
if not ingreso:
    print("Cuenta bloqueada")
```

Salida de ejemplo (entradas: pepe/x, caro/mal, caro/web2026)

```
Usuario: Contraseña: Usuario inexistente
Usuario: Contraseña: Contraseña incorrecta
Usuario: Contraseña: Bienvenido/a, caro
```

24. Roles y permisos

Temas: diccionarios, tuplas, in, condicionales.

Cada rol tiene una tupla de permisos. Para cada solicitud (usuario, acción), buscá el rol del usuario y mostrá "PERMITIDO" o "DENEGADO". Si el usuario no está registrado, denegá con un mensaje distinto.

Datos de partida

```
permisos = {"admin": ("ver", "crear", "editar", "borrar"), "editor": ("ver", "crear", "editar")
    , "lector": ("ver",)}
usuarios = {"ana": "admin", "luis": "editor", "juan": "lector"}
solicitudes = [("luis", "borrar"), ("ana", "borrar"), ("juan", "ver"), ("juan", "crear"), ("pep
    e", "ver")]
```

Resolución

```
permisos = {"admin": ("ver", "crear", "editar", "borrar"), "editor": ("ver", "crear", "editar")
    , "lector": ("ver",)}
usuarios = {"ana": "admin", "luis": "editor", "juan": "lector"}
solicitudes = [("luis", "borrar"), ("ana", "borrar"), ("juan", "ver"), ("juan", "crear"), ("pep
    e", "ver")]
for usuario, accion in solicitudes:
    if usuario not in usuarios:
        print(usuario, accion, "-> DENEGADO (usuario no registrado)")
    elif accion in permisos[usuarios[usuario]]:
        print(usuario, accion, "-> PERMITIDO")
    else:
        print(usuario, accion, "-> DENEGADO")
```

Salida esperada

```
luis borrar -> DENEGADO
ana borrar -> PERMITIDO
juan ver -> PERMITIDO
juan crear -> DENEGADO
pepe ver -> DENEGADO (usuario no registrado)
```

25. Filtrar la respuesta de una API

Temas: lista de diccionarios, for, condicionales.

Una API devolvió esta lista de usuarios. Armá una nueva lista solo con los activos, donde cada elemento sea un diccionario con `nombre` y `email`. Mostrá la nueva lista y cuántos inactivos se descartaron.

Datos de partida

```
respuesta = [
    {"id": 1, "nombre": "Ana", "email": "ana@mail.com", "activo": True, "edad": 22},
    {"id": 2, "nombre": "Luis", "email": "luis@mail.com", "activo": False, "edad": 30},
    {"id": 3, "nombre": "Caro", "email": "caro@mail.com", "activo": True, "edad": 27},
    {"id": 4, "nombre": "Juan", "email": "juan@mail.com", "activo": False, "edad": 19},
]
```

Resolución

```
respuesta = [
    {"id": 1, "nombre": "Ana", "email": "ana@mail.com", "activo": True, "edad": 22},
    {"id": 2, "nombre": "Luis", "email": "luis@mail.com", "activo": False, "edad": 30},
    {"id": 3, "nombre": "Caro", "email": "caro@mail.com", "activo": True, "edad": 27},
    {"id": 4, "nombre": "Juan", "email": "juan@mail.com", "activo": False, "edad": 19},
]
activos = []
descartados = 0
for u in respuesta:
    if u["activo"]:
        activos.append({"nombre": u["nombre"], "email": u["email"]})
    else:
        descartados += 1
print(activos)
print("Inactivos descartados:", descartados)
```

Salida esperada

```
[{'nombre': 'Ana', 'email': 'ana@mail.com'}, {'nombre': 'Caro', 'email': 'caro@mail.com'}]
Inactivos descartados: 2
```

26. Paginación de resultados

Temas: listas, slicing, range, división entera.

Una página muestra 5 artículos por vez. Calculá cuántas páginas hay en total y mostrá los artículos de la página que pida el usuario con `input()`. Si la página no existe, mostrá "Página fuera de rango".

Datos de partida

```
articulos = ["Artículo " + str(i) for i in range(1, 24)]  # 23 artículos
por_pagina = 5
```

Resolución

```
articulos = ["Artículo " + str(i) for i in range(1, 24)]
por_pagina = 5
total_paginas = len(articulos) // por_pagina
if len(articulos) % por_pagina != 0:
    total_paginas += 1
print("Páginas:", total_paginas)
pagina = int(input("Página: "))
if 1 <= pagina <= total_paginas:
    inicio = (pagina - 1) * por_pagina
    for a in articulos[inicio:inicio + por_pagina]:
        print(a)
else:
    print("Página fuera de rango")
```

Salida de ejemplo (entradas: 5)

```
Páginas: 5
Página: Artículo 21
Artículo 22
Artículo 23
```

27. Contador de visitas

Temas: listas, diccionarios, for.

A partir de las páginas visitadas, contá las visitas de cada una con un diccionario y mostrá la página más visitada. Extra: mostrá el top 3 ordenado de mayor a menor.

Datos de partida

```
visitas = ["/", "/productos", "/", "/contacto", "/productos", "/", "/login", "/productos", "/",
     "/blog"]
```

Resolución

El top 3 usa sorted con key=lambda; si no vieron lambda, puede resolverse buscando el máximo tres veces.

```
visitas = ["/", "/productos", "/", "/contacto", "/productos", "/", "/login", "/productos", "/",
     "/blog"]
conteo = {}
for p in visitas:
    conteo[p] = conteo.get(p, 0) + 1
mas_visitada = ""
maximo = 0
for pagina, cant in conteo.items():
    if cant > maximo:
        maximo = cant
        mas_visitada = pagina
print(conteo)
print("Más visitada:", mas_visitada, maximo)
ranking = sorted(conteo.items(), key=lambda x: x[1], reverse=True)
print("Top 3:", ranking[:3])
```

Salida esperada

```
{'/': 4, '/productos': 3, '/contacto': 1, '/login': 1, '/blog': 1}
Más visitada: / 4
Top 3: [('/', 4), ('/productos', 3), ('/contacto', 1)]
```

28. Generar un menú HTML

Temas: lista de tuplas, for, cadenas.

Con un `for`, generá el código HTML de un menú `<ul>` donde cada ítem sea `<li><a href="url">texto</a></li>`. Al ítem cuya URL coincida con `pagina_actual`, agregale `class="activo"` al `<a>`.

Datos de partida

```
menu = [("Inicio", "/"), ("Productos", "/productos"), ("Nosotros", "/nosotros"), ("Contacto", "
    /contacto")]
pagina_actual = "/productos"
```

Resolución

```
menu = [("Inicio", "/"), ("Productos", "/productos"), ("Nosotros", "/nosotros"), ("Contacto", "
    /contacto")]
pagina_actual = "/productos"
html = "<ul>\n"
for texto, url in menu:
    if url == pagina_actual:
        html += f'  <li><a href="{url}" class="activo">{texto}</a></li>\n'
    else:
        html += f'  <li><a href="{url}">{texto}</a></li>\n'
html += "</ul>"
print(html)
```

Salida esperada

```
<ul>
  <li><a href="/">Inicio</a></li>
  <li><a href="/productos" class="activo">Productos</a></li>
  <li><a href="/nosotros">Nosotros</a></li>
  <li><a href="/contacto">Contacto</a></li>
</ul>
```

29. Contraseña nueva hasta que sea válida

Temas: while, input, cadenas, condicionales.

Pedí una contraseña nueva con `input()` y repetí el pedido hasta que cumpla: mínimo 8 caracteres, al menos un número y que no contenga la palabra "password" (sin importar mayúsculas). Después pedí confirmarla y verificá que coincidan. Al final, mostrá cuántos intentos necesitó.

Resolución

```
intentos = 0
valida = False
while not valida:
    clave = input("Nueva contraseña: ")
    intentos += 1
    tiene_numero = False
    for c in clave:
        if c.isdigit():
            tiene_numero = True
    if len(clave) < 8:
        print("Mínimo 8 caracteres")
    elif not tiene_numero:
        print("Debe tener un número")
    elif "password" in clave.lower():
        print("No puede contener 'password'")
    else:
        valida = True
confirmacion = input("Confirmá la contraseña: ")
if confirmacion == clave:
    print("Contraseña guardada en", intentos, "intentos")
else:
    print("No coinciden")
```

Salida de ejemplo (entradas: abc, password123, Comahue2026 y confirmación Comahue2026)

```
Nueva contraseña: Mínimo 8 caracteres
Nueva contraseña: No puede contener 'password'
Nueva contraseña: Confirmá la contraseña: Contraseña guardada en 3 intentos
```

30. Resultados de una encuesta online

Temas: listas, diccionarios, for, porcentajes.

Contá los votos por opción y mostrá el porcentaje de cada una con un decimal. Ignorá los votos que no estén entre las opciones válidas e informá cuántos fueron.

Datos de partida

```
opciones = ("React", "Vue", "Angular", "Svelte")
votos = ["React", "Vue", "React", "Angular", "React", "jQuery", "Vue", "Svelte", "React", "", "
    Vue"]
```

Resolución

```
opciones = ("React", "Vue", "Angular", "Svelte")
votos = ["React", "Vue", "React", "Angular", "React", "jQuery", "Vue", "Svelte", "React", "", "
    Vue"]
conteo = {}
for o in opciones:
    conteo[o] = 0
invalidos = 0
for v in votos:
    if v in opciones:
        conteo[v] += 1
    else:
        invalidos += 1
validos = len(votos) - invalidos
for o, cant in conteo.items():
    print(f"{o}: {cant * 100 / validos:.1f}%")
print("Votos inválidos:", invalidos)
```

Salida esperada

```
React: 44.4%
Vue: 33.3%
Angular: 11.1%
Svelte: 11.1%
Votos inválidos: 2
```

31. Parámetros de una URL

Temas: cadenas, split(), diccionarios, for.

Convertí la parte de parámetros de la URL en un diccionario. Después, si existe el parámetro `edad`, convertilo a entero e indicá si es mayor de edad.

Datos de partida

```
url = "https://tienda.com/buscar?producto=notebook&marca=lenovo&edad=20&orden=precio"
```

Resolución

```
url = "https://tienda.com/buscar?producto=notebook&marca=lenovo&edad=20&orden=precio"
consulta = url.split("?")[1]
parametros = {}
for par in consulta.split("&"):
    clave, valor = par.split("=")
    parametros[clave] = valor
print(parametros)
if "edad" in parametros:
    edad = int(parametros["edad"])
    print("Mayor de edad" if edad >= 18 else "Menor de edad")
```

Salida esperada

```
{'producto': 'notebook', 'marca': 'lenovo', 'edad': '20', 'orden': 'precio'}
Mayor de edad
```

32. Reserva de turnos online

Temas: diccionario de listas, in, condicionales.

Cada día tiene una lista de horarios ocupados. Procesá cada solicitud: si el día no existe, informalo; si el horario está ocupado, rechazalo; si está libre, agregalo a la lista. Mostrá la agenda final.

Datos de partida

```
agenda = {"lunes": ["09:00", "10:00"], "martes": ["11:00"], "miércoles": []}
solicitudes = [("lunes", "09:00"), ("lunes", "11:00"), ("jueves", "10:00"), ("miércoles", "15:0
    0")]
```

Resolución

```
agenda = {"lunes": ["09:00", "10:00"], "martes": ["11:00"], "miércoles": []}
solicitudes = [("lunes", "09:00"), ("lunes", "11:00"), ("jueves", "10:00"), ("miércoles", "15:0
    0")]
for dia, hora in solicitudes:
    if dia not in agenda:
        print(dia, ": no hay atención")
    elif hora in agenda[dia]:
        print(dia, hora, ": ocupado")
    else:
        agenda[dia].append(hora)
        print(dia, hora, ": reservado")
print(agenda)
```

Salida esperada

```
lunes 09:00 : ocupado
lunes 11:00 : reservado
jueves : no hay atención
miércoles 15:00 : reservado
{'lunes': ['09:00', '10:00', '11:00'], 'martes': ['11:00'], 'miércoles': ['15:00']}
```

33. Expiración de sesiones

Temas: lista de tuplas, condicionales compuestos.

Cada tupla es (usuario, rol, minutos de inactividad). La sesión expira con más de 30 minutos, salvo para admin, que expira con más de 60. Separá las sesiones en dos listas: activas y expiradas, y mostralas.

Datos de partida

```
sesiones = [("ana", "admin", 45), ("luis", "cliente", 12), ("caro", "cliente", 31), ("juan", "a
    dmin", 75), ("sol", "editor", 30)]
```

Resolución

```
sesiones = [("ana", "admin", 45), ("luis", "cliente", 12), ("caro", "cliente", 31), ("juan", "a
    dmin", 75), ("sol", "editor", 30)]
activas = []
expiradas = []
for usuario, rol, minutos in sesiones:
    if (rol == "admin" and minutos > 60) or (rol != "admin" and minutos > 30):
        expiradas.append(usuario)
    else:
        activas.append(usuario)
print("Activas:", activas)
print("Expiradas:", expiradas)
```

Salida esperada

```
Activas: ['ana', 'luis', 'sol']
Expiradas: ['caro', 'juan']
```

34. Códigos de estado HTTP

Temas: listas, diccionarios, for, división entera.

Clasificá los códigos de respuesta del servidor en un diccionario por categoría: éxito (2xx), redirección (3xx), error del cliente (4xx) y error del servidor (5xx). Si los errores 5xx superan el 20 % del total, mostrá "Revisar el servidor".

Datos de partida

```
codigos = [200, 200, 404, 301, 500, 200, 403, 502, 200, 304, 503, 200]
```

Resolución

Hay 3 errores 5xx sobre 12 respuestas (25 %), así que corresponde la alerta.

```
codigos = [200, 200, 404, 301, 500, 200, 403, 502, 200, 304, 503, 200]
nombres = {2: "éxito", 3: "redirección", 4: "error del cliente", 5: "error del servidor"}
categorias = {"éxito": 0, "redirección": 0, "error del cliente": 0, "error del servidor": 0}
for c in codigos:
    categorias[nombres[c // 100]] += 1
print(categorias)
if categorias["error del servidor"] / len(codigos) > 0.2:
    print("Revisar el servidor")
```

Salida esperada

```
{'éxito': 5, 'redirección': 2, 'error del cliente': 2, 'error del servidor': 3}
Revisar el servidor
```

Bloque C: códigos que no corren (35 a 50)

Consigna común para todo el bloque: ejecutá el código, copiá el error que muestra Python (o explicá por qué el resultado está mal), explicá la causa con tus palabras y entregá la versión corregida funcionando.

35. Reinicio de servicios

Temas: sintaxis de for e if

Código a revisar

```
servicios = ["apache2", "mysql", "ssh"]
for s in servicios
    print("Reiniciando", s)
if len(servicios) > 2
    print("Reinicio masivo")
```

Resolución

Qué muestra Python

```
SyntaxError: expected ':'
```

Causa: Faltan los dos puntos (:) al final de la línea del for y de la del if.

Código corregido

```
servicios = ["apache2", "mysql", "ssh"]
for s in servicios:
    print("Reiniciando", s)
if len(servicios) > 2:
    print("Reinicio masivo")
```

Salida esperada

```
Reiniciando apache2
Reiniciando mysql
Reiniciando ssh
Reinicio masivo
```

36. Cambiar el puerto del servidor

Temas: tuplas (inmutabilidad)

Código a revisar

```
servidor = ("192.168.1.10", 22, "ubuntu")
print("Conectando a", servidor[0], "puerto", servidor[1])
servidor[1] = 2222
print("Nuevo puerto:", servidor[1])
```

Resolución

Qué muestra Python

```
TypeError: 'tuple' object does not support item assignment
```

Causa: Las tuplas son inmutables: no se puede cambiar un elemento. Hay que crear una tupla nueva (o usar una lista).

Código corregido

```
servidor = ("192.168.1.10", 22, "ubuntu")
print("Conectando a", servidor[0], "puerto", servidor[1])
servidor = (servidor[0], 2222, servidor[2])
print("Nuevo puerto:", servidor[1])
```

Salida esperada

```
Conectando a 192.168.1.10 puerto 22
Nuevo puerto: 2222
```

37. Mensaje de fin de instalación

Temas: tipos de datos, concatenación

Código a revisar

```
pcs = ["pc01", "pc02", "pc03"]
cantidad = len(pcs)
print("Se configuraron " + cantidad + " equipos")
```

Resolución

Qué muestra Python

```
TypeError: can only concatenate str (not "int") to str
```

Causa: Se intenta sumar texto con un entero. Hay que convertir el número con str() o pasar los valores separados por coma a print().

Código corregido

```
pcs = ["pc01", "pc02", "pc03"]
cantidad = len(pcs)
print("Se configuraron " + str(cantidad) + " equipos")
```

Salida esperada

```
Se configuraron 3 equipos
```

38. Revisión de servicios

Temas: diccionarios, claves inexistentes

Además de corregirlo, hacé que para un servicio desconocido muestre "no instalado".

Código a revisar

```
estado = {"apache2": "activo", "mysql": "inactivo"}
revisar = ["apache2", "mysql", "nginx"]
for s in revisar:
    print(s, "->", estado[s])
```

Resolución

Qué muestra Python

```
KeyError: 'nginx'
```

Causa: La clave "nginx" no existe en el diccionario. Se usa get() con un valor por defecto (o se verifica antes con in).

Código corregido

```
estado = {"apache2": "activo", "mysql": "inactivo"}
revisar = ["apache2", "mysql", "nginx"]
for s in revisar:
    print(s, "->", estado.get(s, "no instalado"))
```

Salida esperada

```
apache2 -> activo
mysql -> inactivo
nginx -> no instalado
```

39. Instalación de paquetes por índice

Temas: listas, range, índices

Código a revisar

```
paquetes = ["git", "curl", "vim"]
for i in range(len(paquetes) + 1):
    print("Instalando", paquetes[i])
print("Instalación terminada")
```

Resolución

Qué muestra Python

```
IndexError: list index out of range
```

Causa: range(len(paquetes) + 1) llega al índice 3, que no existe (los índices van de 0 a 2).

Código corregido

```
paquetes = ["git", "curl", "vim"]
for i in range(len(paquetes)):
    print("Instalando", paquetes[i])
print("Instalación terminada")
```

Salida esperada

```
Instalando git
Instalando curl
Instalando vim
Instalación terminada
```

40. Reintentos que nunca terminan

Temas: while, condición de corte

Atención: este programa no termina solo; cortalo con Ctrl+C. Corregilo para que haga exactamente 3 intentos.

Código a revisar

```
intentos = 0
while intentos < 3:
    print("Intentando conectar...")
print("Fin")
```

Resolución

Qué muestra Python

```
No termina nunca: imprime "Intentando conectar..." sin parar (bucle infinito).
```

Causa: La variable intentos nunca cambia, así que la condición siempre es verdadera (bucle infinito). Hay que incrementarla dentro del while.

Código corregido

```
intentos = 0
while intentos < 3:
    print("Intentando conectar...")
    intentos += 1
print("Fin")
```

Salida esperada

```
Intentando conectar...
Intentando conectar...
Intentando conectar...
Fin
```

41. Alta de usuarios

Temas: indentación

Código a revisar

```
usuarios = ["ana", "luis"]
for u in usuarios:
print("Creando usuario", u)
  print("Listo")
```

Resolución

Qué muestra Python

```
IndentationError: expected an indented block after 'for' statement on line 2
```

Causa: El cuerpo del for no está indentado y la última línea tiene una sangría que no corresponde a ningún bloque.

Código corregido

```
usuarios = ["ana", "luis"]
for u in usuarios:
    print("Creando usuario", u)
print("Listo")
```

Salida esperada

```
Creando usuario ana
Creando usuario luis
Listo
```

42. Control de acceso por rol

Temas: condicionales, operadores de comparación

Código a revisar

```
rol = "admin"
if rol = "admin":
    print("Acceso total")
else:
    print("Acceso restringido")
```

Resolución

Qué muestra Python

```
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

Causa: = es asignación; para comparar se usa ==.

Código corregido

```
rol = "admin"
if rol == "admin":
    print("Acceso total")
else:
    print("Acceso restringido")
```

Salida esperada

```
Acceso total
```

43. Listado de puertos por servicio

Temas: recorrer diccionarios

Código a revisar

```
puertos = {"ssh": 22, "http": 80, "mysql": 3306}
for servicio, puerto in puertos:
    print(servicio, "usa el puerto", puerto)
```

Resolución

Qué muestra Python

```
ValueError: too many values to unpack (expected 2)
```

Causa: Recorrer un diccionario devuelve solo las claves; Python intenta repartir las letras de "ssh" en dos variables. Hay que usar .items().

Código corregido

```
puertos = {"ssh": 22, "http": 80, "mysql": 3306}
for servicio, puerto in puertos.items():
    print(servicio, "usa el puerto", puerto)
```

Salida esperada

```
ssh usa el puerto 22
http usa el puerto 80
mysql usa el puerto 3306
```

44. Registro con edad mínima

Temas: input, conversión de tipos, condicionales

Código a revisar

```
edad = input("Ingresá tu edad: ")
if edad >= 18:
    print("Podés registrarte")
else:
    print("Sos menor de edad")
```

Resolución

Qué muestra Python

```
TypeError: '>=' not supported between instances of 'str' and 'int'
```

Causa: input() siempre devuelve texto, y no se puede comparar texto con un número. Hay que convertir con int().

Código corregido

```
edad = int(input("Ingresá tu edad: "))
if edad >= 18:
    print("Podés registrarte")
else:
    print("Sos menor de edad")
```

Salida esperada (entrada: 20)

```
Ingresá tu edad: Podés registrarte
```

45. Limpieza de sesiones expiradas

Temas: listas, modificar una lista mientras se recorre

Este código no da error, pero el resultado está mal: deberían quedar solo las sesiones activas. Explicá por qué pasa y corregí.

Código a revisar

```
sesiones = ["expirada", "expirada", "activa", "expirada", "activa"]
for s in sesiones:
    if s == "expirada":
        sesiones.remove(s)
print(sesiones)
```

Resolución

Qué muestra Python

```
['activa', 'expirada', 'activa']
```

Causa: Al borrar elementos mientras se recorre la lista, los siguientes se corren de posición y el for se saltea algunos. Conviene armar una lista nueva (o recorrer una copia: sesiones[:]).

Código corregido

```
sesiones = ["expirada", "expirada", "activa", "expirada", "activa"]
activas = []
for s in sesiones:
    if s != "expirada":
        activas.append(s)
print(activas)
```

Salida esperada

```
['activa', 'activa']
```

46. Total de una compra

Temas: variables, acumuladores

Código a revisar

```
precios = [1500, 2300, 800]
total = 0
for p in precios:
    total = total + p
print("Total:", Total)
```

Resolución

Qué muestra Python

```
NameError: name 'Total' is not defined. Did you mean: 'total'?
```

Causa: Python distingue mayúsculas de minúsculas: Total y total son variables distintas, y Total nunca se definió.

Código corregido

```
precios = [1500, 2300, 800]
total = 0
for p in precios:
    total = total + p
print("Total:", total)
```

Salida esperada

```
Total: 4600
```

47. Agregar productos al carrito

Temas: listas, tuplas, append

Corregilo de forma que el carrito guarde cada producto junto con su precio.

Código a revisar

```
carrito = []
carrito.append("mouse", 9500)
carrito.append("teclado", 18000)
print(carrito)
```

Resolución

Qué muestra Python

```
TypeError: list.append() takes exactly one argument (2 given)
```

Causa: append() recibe un solo elemento. Para guardar producto y precio juntos se agrega una tupla.

Código corregido

```
carrito = []
carrito.append(("mouse", 9500))
carrito.append(("teclado", 18000))
print(carrito)
```

Salida esperada

```
[('mouse', 9500), ('teclado', 18000)]
```

48. Total vendido del día

Temas: for, acumuladores

No da error, pero muestra un total incorrecto (debería ser 47500).

Código a revisar

```
ventas = [12000, 8500, 23000, 4000]
for v in ventas:
    total = 0
    total += v
print("Total vendido:", total)
```

Resolución

Qué muestra Python

```
Total vendido: 4000
```

Causa: El acumulador se vuelve a 0 en cada vuelta del for, así que al final solo queda la última venta. Hay que inicializarlo antes del for.

Código corregido

```
ventas = [12000, 8500, 23000, 4000]
total = 0
for v in ventas:
    total += v
print("Total vendido:", total)
```

Salida esperada

```
Total vendido: 47500
```

49. Verificar la versión de Python antes de instalar

Temas: comparación de cadenas, tuplas

No da error, pero dice que hay que actualizar cuando la versión 3.10 es más nueva que la 3.9. Explicá por qué y corregí usando tuplas de enteros.

Código a revisar

```
version_instalada = "3.10"
version_minima = "3.9"
if version_instalada >= version_minima:
    print("Versión compatible")
else:
    print("Actualizar Python")
```

Resolución

Qué muestra Python

```
Actualizar Python
```

Causa: Las cadenas se comparan carácter a carácter: en la tercera posición "1" es menor que "9". Convirtiendo a tuplas de enteros se compara (3, 10) con (3, 9), que es lo correcto.

Código corregido

```
version_instalada = "3.10"
version_minima = "3.9"
instalada = tuple(int(x) for x in version_instalada.split("."))
minima = tuple(int(x) for x in version_minima.split("."))
if instalada >= minima:
    print("Versión compatible")
else:
    print("Actualizar Python")
```

Salida esperada

```
Versión compatible
```

50. Mejor puntaje

Temas: listas, sort() y sorted()

Código a revisar

```
puntajes = [70, 95, 88, 60]
ordenados = puntajes.sort()
print("Mejor puntaje:", ordenados[-1])
```

Resolución

Qué muestra Python

```
TypeError: 'NoneType' object is not subscriptable
```

Causa: sort() ordena la lista en el lugar y devuelve None. Para obtener una lista ordenada nueva se usa sorted().

Código corregido

```
puntajes = [70, 95, 88, 60]
ordenados = sorted(puntajes)
print("Mejor puntaje:", ordenados[-1])
```

Salida esperada

```
Mejor puntaje: 95
```